

# Custom Events Workflow

## Technical Guide

---

Event Visualisation & Status Grouping — Methodology & Calculation Reference

Version 0.1.4 · Generated June 29, 2026

---

## 1. Overview

The **Custom Events Workflow** downloads EarthRanger events for one or more event types over a selected time period, classifies each event into a status group (Active, Resolved, or All), and delivers an interactive four-widget dashboard — an Events Map, an Events Bar Chart, an Events Pie Chart, and an Event Count Map.

The key distinguishing feature is **result-time status grouping**: all three status views are computed in a single run and stored together in the dashboard. The user switches between views using a dropdown in the View panel without re-running the workflow.

The workflow connects to **EarthRanger** via a configured data source. Events are filtered by event type, time range, and optionally by a geographic bounding box. All charts and maps share a consistent colour scheme keyed on event type.

## 2. Dependencies & Prerequisites

### 2.1 EarthRanger Connection

All event data is fetched from an **EarthRanger** instance via `set_er_connection`. Events are retrieved with `get_events` over the configured time range.

Key fetch parameters:

| Parameter                           | Value                                                                                          |
|-------------------------------------|------------------------------------------------------------------------------------------------|
| <code>event_columns</code>          | <code>id, time, event_type, event_category, reported_by, serial_number, state, geometry</code> |
| <code>raise_on_empty</code>         | <code>false</code> — workflow continues gracefully when no events exist for the selection      |
| <code>include_details</code>        | <code>false</code>                                                                             |
| <code>include_display_values</code> | <code>true</code> — populates the <code>event_type_display</code> human-readable column        |

### 2.2 Event Type Resolver

On **Ecoscope Web**, the `event_types` field uses the `EarthRangerEnumResolver` transformer to populate a live dropdown from the connected EarthRanger instance. On **Ecoscope Desktop**, event type identifiers (e.g. `hwc_rep`, `fire_rep`) are entered manually. Identifiers are found in EarthRanger under **Admin** → **Event Types**.

### 2.3 Grouper

One grouper field drives the dashboard's multi-view structure:

| Index name                      | Column                                          | Effect                                                     |
|---------------------------------|-------------------------------------------------|------------------------------------------------------------|
| <code>event_status_group</code> | <code>Computed by add_event_status_group</code> | One dashboard tab per status group (Active, Resolved, All) |

The grouper is wired into `set_groupers` and passed through the entire downstream pipeline via `resolved_groupers`.

### 2.4 Base Map Tile Layers

Base maps are configured via `set_base_maps` and composited beneath all event map layers. The default preset is a hybrid of Terrain (World Topo Map) at full opacity and Satellite imagery at 50% opacity.

### 3. Data Ingestion Pipeline

#### 3.1 Events Fetch

`get_events` retrieves all events matching the selected event types and time range. The raw response includes the `state` field (e.g. `active`, `done`, `resolved`) and the `reported_by` JSON object, which is unpacked in a subsequent step.

#### 3.2 Timezone Handling

`get_timezone_from_time_range` extracts the IANA timezone string from the configured time range. `convert_values_to_timezone` converts the time column from UTC to local time. The display format used throughout the dashboard is `%d %b %Y %H:%M:%S`.

#### 3.3 Reporter Extraction

`extract_value_from_json_column` unpacks the `reported_by` JSON column, extracting the `name` field into a new `reported_by_name` string column. This column is used as the **Reported By** tooltip field on the Events Map.

### 4. Event Status Grouping

This is the custom logic that distinguishes this workflow from the standard events workflow.

#### 4.1 Algorithm

`add_event_status_group` (implemented in `custom-events-tasks`) adds an `event_status_group` column to the events `GeoDataFrame` and then **duplicates every row** into an "All" group. The result has up to two rows per original event:

- One row with its mapped status group (Active or Resolved)
- One row with `event_status_group = "All"`

This duplication means that when the user selects **All** in the dashboard dropdown, every event is visible — not a separate filtered query.

#### 4.2 State-to-Group Mapping

| EarthRanger state | event_status_group |
|-------------------|--------------------|
| active            | Active             |
| scheduled         | Active             |
| overdue           | Active             |
| done              | Resolved           |
| resolved          | Resolved           |
| cancelled         | Resolved           |
| (any other value) | Unknown            |

**Note:** If the state column is absent from the data, all events are assigned Unknown.

#### 4.3 Dashboard Grouper Integration

After `add_event_status_group`, `set_groupers` configures `event_status_group` as the sole grouper. `split_groups` partitions the `GeoDataFrame` into one slice per distinct value of `event_status_group`. Each downstream branch (bar chart, map, pie chart, event count map) receives and processes these slices independently, producing one widget view per status group. The `merge_widget_views` step combines them into a single multi-view widget that the dashboard renders as a dropdown switcher.

## 5. Event Processing Pipeline

### 5.1 Location Filtering

`apply_reloc_coord_filter` filters events to a geographic bounding box and optional coordinate exclusion polygons. The filter is applied after `add_event_status_group` so grouping is preserved regardless of spatial filtering. Default bounds (−180 to 180, −90 to 90) pass all events.

### 5.2 Temporal and Spatial Index

`add_temporal_index` and `add_spatial_index` attach grouper-aware index columns to the filtered events. Both steps reference `resolved_groupers`, which carries the `event_status_group` grouper through after spatial feature group resolution.

### 5.3 Colour Mapping

`apply_color_map` maps the `event_type` column to a hex colour string using the `tab10` palette, storing the result in `event_type_colormap`. This column drives the fill colour on the Events Map and the bar and pie chart segments. All four dashboard widgets share this colour scheme.

### 5.4 Column Renaming for Tooltip Display

Before the Events Map is drawn, `map_columns` renames internal column names to human-readable display labels:

| Internal column                 | Display label |
|---------------------------------|---------------|
| <code>serial_number</code>      | Event Serial  |
| <code>time</code>               | Event Time    |
| <code>event_type_display</code> | Event Type    |
| <code>reported_by_name</code>   | Reported By   |

**Note:** `raise_if_not_found: true` — the workflow will fail if any of these columns is missing, ensuring tooltip configuration is always consistent.

## 6. Dashboard Outputs

### 6.1 Events Map

Produced by `create_point_layer` → `draw_ecomap` → `persist_text` → `create_map_widget_single_view` → `merge_widget_views`.

| Property         | Value                                                 |
|------------------|-------------------------------------------------------|
| Layer type       | Point                                                 |
| Fill colour      | event_type_colormap column                            |
| Point radius     | 5 px                                                  |
| Tooltip columns  | Event Serial, Event Time, Event Type, Reported By     |
| Legend           | Label: Event Type column; Colour: event_type_colormap |
| Legend placement | Bottom-right                                          |
| North arrow      | Top-left                                              |
| Static           | false (interactive: pan, zoom, click tooltips)        |
| Max zoom         | 20                                                    |

**Note:** The `all_geometry_are_none` skip condition cleanly skips the point layer and ecomap steps when a status group has all-null geometry. Widget tasks use `skipif`: never so the dashboard always assembles.

## 6.2 Events Bar Chart

Produced by `draw_time_series_bar_chart` → `persist_text` → `create_plot_widget_single_view` → `merge_widget_views`.

| Property       | Value                                                       |
|----------------|-------------------------------------------------------------|
| X axis         | time (at user-selected interval: day / week / month / year) |
| Y axis         | event_type_display (aggregated as count)                    |
| Category       | event_type_display                                          |
| Aggregation    | count                                                       |
| Colour column  | event_type_colormap                                         |
| Y axis title   | Count of Events by Type                                     |
| X axis title   | Time                                                        |
| X period align | middle                                                      |

## 6.3 Events Pie Chart

Produced by `draw_pie_chart` → `persist_text` → `create_plot_widget_single_view` → `merge_widget_views`.

| Property      | Value                             |
|---------------|-----------------------------------|
| Value column  | event_type_display                |
| Colour column | event_type_colormap               |
| Text info     | value (count shown on each slice) |

## 6.4 Event Count Map (Density Heatmap)

Pipeline: `create_meshgrid` → `calculate_feature_density` → `sort_values` → `drop_nan_values_by_column` → `apply_classification` → `apply_color_map` → `map_columns` → `create_polygon_layer` → `draw_ecomap` → `widget`.

| Step             | Detail                                                                                                           |
|------------------|------------------------------------------------------------------------------------------------------------------|
| Grid             | <code>create_meshgrid</code> — auto-scaled to AOI extent by default, or fixed cell size in metres                |
| Density          | <code>calculate_feature_density</code> — counts point features per grid cell; <code>geometry_type</code> : point |
| Classification   | <code>apply_classification</code> — equal interval, 10 classes; labels show integer-rounded ranges               |
| Colour palette   | <code>RdYlGn_r</code> (red = high density, green = low density)                                                  |
| Fill opacity     | 0.40                                                                                                             |
| Line width       | 0 (no cell borders)                                                                                              |
| Tooltip          | Count — the number of events in the cell                                                                         |
| Zero-count cells | Dropped via <code>drop_nan_values_by_column</code> before colouring — transparent in the map                     |

**Note:** The grid is computed once on the full (pre-split) `GeoDataFrame` and reused across all status group slices, so grid resolution is consistent across Active, Resolved, and All views.

## 7. Interactive Dashboard

`gather_dashboard` assembles the final dashboard from four widget groups, all bound to the `event_status_group` grouper and the configured time range:

| Widget           | Type | Pipeline                                                                             |
|------------------|------|--------------------------------------------------------------------------------------|
| Events Bar Chart | Plot | <code>events_bar_chart</code> → <code>grouped_bar_plot_widget_merge</code>           |
| Events Map       | Map  | <code>grouped_events_ecomap</code> → <code>grouped_events_map_widget_merge</code>    |
| Events Pie Chart | Plot | <code>grouped_events_pie_chart</code> → <code>grouped_events_pie_widget_merge</code> |
| Event Count Map  | Map  | <code>grouped_fd_ecomap</code> → <code>grouped_fd_map_widget_merge</code>            |

The dashboard layout places widgets in a two-column grid: Events Map (left) / Events Bar Chart (right) / Event Count Map (left) / Events Pie Chart (right).

The **Event status group** dropdown in the View panel switches all four widgets simultaneously between Active, Resolved, and All — no re-run required.

## 8. Output Files

All files are written to `$ECOSCOPE_WORKFLOWS_RESULTS`.

| File pattern                      | Type | Description                                          |
|-----------------------------------|------|------------------------------------------------------|
| *_v2.html (events map, per group) | HTML | Interactive ecomap — Events Map per status group     |
| *_v2.html (bar chart, per group)  | HTML | Plotly bar chart — Events Bar Chart per status group |
| *_v2.html (pie chart, per group)  | HTML | Plotly pie chart — Events Pie Chart per status group |

| File pattern                           | Type | Description                                           |
|----------------------------------------|------|-------------------------------------------------------|
| *_v2.html (event count map, per group) | HTML | Interactive ecomap — Event Count Map per status group |

## 9. Workflow Execution Logic

### 9.1 Skip Conditions

Two default skip conditions apply to every task (task-instance-defaults):

- **any\_is\_empty\_df** — skips the task (and all dependants) when any input GeoDataFrame is empty. This handles event type selections that return no data for the time period.
- **any\_dependency\_skipped** — propagates skips downstream automatically.

Widget creation tasks override this with skipif: never to ensure the dashboard always assembles, even when a status group contains no events. The `create_point_layer` and `create_polygon_layer` steps add `all_geometry_are_none` as an additional skip condition.

### 9.2 Data Flow Summary

| Stage              | Tasks                                                                                                         |
|--------------------|---------------------------------------------------------------------------------------------------------------|
| Setup              | set_workflow_details, set_er_connection, set_time_range, get_timezone_from_time_range, set_groupers           |
| Data ingest        | get_events → convert_values_to_timezone → extract_value_from_json_column                                      |
| Status grouping    | add_event_status_group (custom task — duplicates rows into Active / Resolved / All)                           |
| Spatial setup      | resolve_spatial_feature_groups_for_spatial_groupers, apply_reloc_coord_filter                                 |
| Indexing           | add_temporal_index → add_spatial_index                                                                        |
| Colouring          | apply_color_map (tab10 by event type)                                                                         |
| Split              | split_groups (one slice per event_status_group value)                                                         |
| Events Map branch  | map_columns (rename) → create_point_layer → draw_ecomap → persist_text → widget                               |
| Bar Chart branch   | draw_time_series_bar_chart → persist_text → widget                                                            |
| Pie Chart branch   | draw_pie_chart → persist_text → widget                                                                        |
| Event Count branch | create_meshgrid → calculate_feature_density → classify → colour → create_polygon_layer → draw_ecomap → widget |
| Dashboard          | gather_dashboard combines all four merged widget groups                                                       |

## 10. Software Versions

| Package             | Version | Role                                                                 |
|---------------------|---------|----------------------------------------------------------------------|
| ecoscope-platform   | 2.13.0  | Core task library, workflow engine, spatial analysis                 |
| custom-events-tasks | 0.1.4   | Bundled custom tasks: add_event_status_group, filter_events_by_state |

| Package | Version | Role                                                                                    |
|---------|---------|-----------------------------------------------------------------------------------------|
| pydeck  | 0.9.2   | Map rendering (pinned — re-declared as workaround for known pydeck dependency conflict) |

The ecoscope-platform package is distributed via the <https://repo.prefix.dev/ecoscope-workflows/> conda channel. The custom-events-tasks package is bundled directly inside the compiled workflow directory (ecoscope-workflows-custom-events-workflow/custom-events-tasks/) and installed as a local editable PyPI package via a relative path in pixi.toml. The runtime environment is managed by **pixi**.